G4011105 – Programación I

<u>Nota</u>: Documentar cada programa indicando autor, data de creación, breve descrición do que fai o programa e datos que precisa. Documentar tamén as principais operacións do algoritmo. Isto hai que facelo para todos os exercicios do curso.

1. Empezaremos vendo como é o proceso de creación dun proxecto, a codificación e a compilación co seguinte exemplo:

```
/*
        Autor : Xosé Outeiro
        Data : 1 de outubro de 2020
        Versión: 2
        Este é programa clacula a media de dous valores reais dados polo usuario
* /
#include<stdio.h>

int main(int argc, char** argv)
{
      float val1,val2,media;

      printf ("Introduza o valor 1: ");
      scanf("%f", &val1);
      printf ("Introduza o valor 2: ");
      scanf("%f", &val2);

      media= (val1 + val2)/2.0;
      printf("Valor medio: %f\n", media)

      return(EXIT_SUCCESS);
}
```

2. Comentaremos agora un pouco a estrutura básica dun programa en C:

```
/*
       Autor : Fernando García
       Data : 7 de abril de 2017
       Versión : 1
       Este programa calcula a área e o perímetro dun circulo, a partires do seu radio
*/
#include<stdio.h>        //Directivas preprocesador
#define PI 3.1416

int main(int argc, char** argv)    //COMEZO PARTE EXECUTIVA
{
      float r,a,p;             //Declaración de variables locais

      // Lectura de datos
      printf ("Introduza o radio do círculo:");
      scanf("%f",&r);

      // Cálculo dos resultados
      p=2*PI*r;
      a=PI*r*r;

      //-------- Saída da información ----------*)
      printf ("Área do círculo  = %f \n",a);
      printf ("Perímetro do círculo = %f \n",p);
      return(EXIT_SUCCESS);
}                              // FIN PARTE EXECUTIVA
```

3. Escribir un programa que dados dous enteiros positivos calcule a división enteira e o resto.

4. Escribir un programa que lea un carácter por teclado e imprima en pantalla o seu código ASCII. Para iso haberá que executar a operación de lectura dunha carácter, despois executar unha operación de escritura dun enteiro (función **printf** con especificador de formatos "**%d"**), pasándolle a variable onde se gardou o carácter anterior. Observar que: como un dato de tipo char se codifica mediante un valor enteiro, podemolo manexar indistintamente como se fose un i**nt** ou un **char**.

5. Escribir un programa que pida unha letra por teclado e que imprima por pantalla a súa maiúscula. Hai que incluír o ficheiro cabeceira **ctype.h** onde está definida a función **toupper()** que devolve o caracter correspondente á maiúscula da letra que se lle pasa como argumento.

   A función toupper() ten o seguinte prototipo:
   ***int toupper(int ch);** // equivalente a: char toupper(char ch)*

   En relación co aprendido no exercicio previo, as funcións que manexan datos de tipo caracter, adoitan ter unha representación do seu prototipo como se recivisen datos de tipo enteiro e devolovesen valores enteiros.

6. Escribir un programa que realice o redondeo dunha cantidade real ao enteiro máis próximo. Hai que incluír a cabeceira **math.h** onde están definidas as funcións matemáticas. Probar coas funcións **floor(), ceil() e round()**, e ver como se comportan con positivos e con negativos.

7. Escribir un programa que lea unha palabra e que nos diga cantas letras ten. Para calcular o número de caracteres podedes usar a funcion **strlen(),** definida en **string.h**, á que lle temos que pasar a cadea de caracteres que contén a palabra e nos devolve o seu tamaño. Imprimide unha mensaxe no que apareza a palabra e o número de letras que contén.

   A funcion strlen() ten o seguinte prototipo:
   ***size_t strlen(const char *str);** //equivalente a int strlen(char str[N])*

   Neste caso *char *str* é equivalente a *char str[N]*, e indica que a esta función hai que pasarlle como argumento un dato de tipo cadea de caracteres. *size_t* é outro nome para representar un enteiro non negativo (unsigned int), que se define nalgún ficheiro cabeceira.

8. Repetir o exercicio anterior, pero agora para o caso dunha frase.